

CareReady

Staff & Device Readiness

For multi-site care organizations. A complete watch, build, test, and commit progression alongside Bookly.

Your product

CareReady helps a small central IT team get equipment ready for staff start dates and recover it safely at placement end. It links sites, staff placements, kit requests, device reservations, approvals, shipments, setup checks, custody, returns, and wipe evidence.

Your workflow

Watch to a listed timestamp. Pause. Build the stated behavior, prove its acceptance checks, then make the suggested commit. Resume only after you can explain the result. Build estimates are flexible; there is no fixed daily deadline. Multiple commits can happen while paused at the same video timestamp.

Two independent repositories

CareReady: github.com/PraneetS11/careready-api Bookly: github.com/PraneetS11/bookly-api

Starting point and honesty

The application foundation, settings, health endpoints, async clients, local services, and initial tests already exist. Domain features below are planned. Do not create empty commits or claim a checkpoint is complete just because you watched it. Use your real commit date and record your actual commit SHA in the progress log.

Reference

Ssali Jonathan: FastAPI Beyond CRUD. Video duration 12:52:54. The full chapter list was verified on 24 September 2026; chapter labels here are paraphrased. CareReady requirements and tests are independent exercises.

<https://www.youtube.com/watch?v=TO4aQ3ghFOc>

The readiness gap CareReady closes

A fictional Canadian care network has six sites and a small IT team. A site coordinator requests a laptop kit for a new hire or rotating placement. A ticket being marked complete is not enough: the device must be allocated, delivered, configured, and acknowledged before the staff member starts.

This connects IT support and workflow automation with practical onboarding. Use fabricated organizations and staff. CareReady is independent of any employer and excludes patient records, clinical decisions, payments, and direct hardware wiping.

Role	Responsibility
Site coordinator	Create placement and kit requests; acknowledge delivery.
IT operator	Reserve stock, complete setup, record custody and returns.
Approver	Review purchases independently of the requester.
Organization admin	Manage memberships, sites, templates, and policy.
Staff member	View own requests and assigned equipment.
Auditor	Read tenant-scoped evidence and event history.

Core scenario

Cedar Care has Hamilton and Toronto sites, two laptops, three placements, a delayed shipment, and an overdue return. Harbour Care is a second isolated tenant. Demonstrate stock shortage, independent approval, readiness blockers, return inspection, and rejected cross-tenant access.

State and invariants

- Readiness: draft -> submitted -> approved -> preparing -> dispatched -> ready. Rejected/cancelled are explicit alternatives. Ready requires delivery and all required setup checks.
- Reuse: opened -> in_transit -> received -> inspected -> wipe_verified -> closed. Only then may the asset become available, unless an authorized exception is recorded.
- No self-approval, stale approval, double reservation, cross-tenant reference, or repeated shipment side effect.

Pause, prove, commit, resume

At the start of a work session

- Open the video, this guide, both repositories, and the local API docs. Run make services, make test, and make lint from the relevant repository.
- Create a feature branch for the next unfinished checkpoint. Read its gate before watching so you know what you must be able to do.
- Use fictional fixtures. Keep early unauthenticated prototypes local until tenant and authorization checks exist.

At each commit checkpoint

- Pause exactly at the listed video position. Build the behavior independently in CareReady, then complete the relevant Bookly parallel.
- Run focused tests and the full existing suite, lint, and formatting checks. Review your diff and document new behavior and limitations.
- Commit in the relevant repository with the suggested message, or an accurate equivalent. Push the working branch and record the SHA.
- If one gate fails, keep the video paused. Split large checkpoints into more real commits when needed; never fabricate progress.

```
git switch -c feat/care-site-contract
```

```
make test && make lint
```

```
git diff
```

```
git add <specific-files>
```

```
git commit -m "feat: add validated care site endpoints"
```

```
git push -u origin HEAD
```

What a commit time means

A time such as 1:38:22 is a video pause marker, not elapsed coding time. Commit after its checks pass, even if that takes another day. The five checkpoints at session 08 include several commits at 7:59:58 because the independent business workflow needs more work than the corresponding watch block.

Already-implemented foundation

Do not recreate setup commits. Begin with C01. Installed libraries are preparation, not completed functionality. Bookly remains a separate product and receives its own meaningful commits. Keep product READMEs focused on product behavior.

Your next 13 stages

Stage / video range	CareReady deliverable
01 / 0:00-0:49:43	HTTP foundations
02 / 0:49:43-1:38:22	CRUD and route boundaries
03 / 1:38:22-2:29:48	Async persistence
04 / 2:29:48-3:33:35	Services and dependency injection
05 / 3:33:35-4:42:57	Users and schema evolution
06 / 4:42:57-6:07:39	Sessions, JWTs, and revocation
07 / 6:07:39-6:39:24	Authorization and tenant context
08 / 6:39:24-7:59:58	Relationships and operational workflows
09 / 7:59:58-9:05:04	Error contracts, logs, and audit evidence
10 / 9:05:04-10:40:38	Email and account recovery
11 / 10:40:38-11:23:48	Durable background work
12 / 11:23:48-12:09:17	API contracts and test depth
13 / 12:09:17-12:52:54	Deployment and operations

Completion gate

Each stage requires its listed acceptance checks, a working commit, and a short explanation in your own words. Test business rules when introduced rather than waiting for the testing chapter. Readiness, reservation, and return stages will take multiple coding sessions.

Tracking

The editable docs/PROGRESS.md checklist in each repository is blank for future features. Record dates, commit SHAs, proof, and the next unresolved question. The repository foundation is complete; none of the future checklist items has been marked done for you.

HTTP foundations

Watch 50 min; build 30-45 min.

Understand

Understand how method, URL, path parameter, query parameter, headers, and JSON body reach a Python function. The existing starter is your launch point, not a reason to skip understanding routing.

Build in CareReady

- Create SiteCreate with name, city, timezone, and active; use fictional Hamilton and Toronto care sites.
- Implement GET /api/v1/sites, GET /api/v1/sites/{id}, and POST /api/v1/sites using temporary memory. Add an active filter.
- Sketch ReadinessRequestCreate with site_id, staff_reference, role_template, start_date, and end_date. Validate end_date >= start_date. The request workflow is implemented later; do not imply persistence or tenant protection yet.

Bookly parallel

Build the corresponding book list/detail/create operations with title, author, ISBN, and publication date.

Prove before moving on

- POST valid data returns 201; malformed data returns 422.
- Unknown ID returns 404; query filtering changes the result.

Explain

What belongs in a path versus a query? Why can validation reject a request before the handler runs?

Commit checkpoints: C01 at 0:49:43. Exact gates and messages are on the next page.

When to commit

C01 / Pause at 0:49:43 / Site HTTP contract

Create/list/detail and active filtering; malformed data 422, missing site 404, placement date schema rejects reversed dates.

CareReady: feat: add validated care site endpoints

Bookly: feat: add validated book endpoints

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

CRUD and route boundaries

Watch 49 min; build 35-60 min.

Understand

Follow list-based REST operations, then route grouping at 1:23:37. Understand HTTP semantics before adding database complexity.

Build in CareReady

- Move site handlers to `app/api/sites.py` and add PATCH and DELETE. Distinguish omitted fields from explicit nulls.
- Add status/city filters and consistent 404/422 responses. Use a fictional staff kit request to explain why site identity matters.
- Document future archive behavior: a site referenced by a placement must not disappear from its audit history. Physical delete is limited to unused prototype records.

Bookly parallel

Organize `/books` in its own router; add edits, deletion behavior, and author filtering.

Prove before moving on

- Missing site returns 404 consistently across read/update/delete.
- Invalid patch preserves prior state. A 204 delete response has no JSON body.

Explain

When is PATCH different from PUT? Which rules belong in schemas versus services?

Commit checkpoints: C02 at 1:23:37, C03 at 1:38:22. Exact gates and messages are on the next page.

When to commit

C02 / Pause at 1:23:37 / Site CRUD

PATCH preserves omitted fields; deletion and missing IDs have documented responses.

CareReady: feat: implement care site CRUD

Bookly: feat: implement book CRUD

C03 / Pause at 1:38:22 / Router separation

Site routes live under /api/v1; filters and all earlier tests still pass.

CareReady: refactor: organize care site API routes

Bookly: refactor: organize catalog API routes

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Async persistence

Watch 51 min; build 75-120 min.

Understand

Slow down for settings, async engine/session setup, lifespan, models, and SQLAlchemy persistence. The starter already owns clients in lifespan; extend that design.

Build in CareReady

- Define a Site SQLAlchemyModel with UUID, organization_id, name, timezone, active, and UTC timestamps. Use a fixed fictional organization until real membership enforcement exists.
- Extend the existing request-scoped async session dependency to persist site creation and listing in PostgreSQL.
- Verify data survives an API restart and failed writes roll back. Understand disposable table creation, then replace it with reviewed Alembic migrations in session 05.

Bookly parallel

Define a Book table, separate request/response schemas, and verify catalog persistence.

Prove before moving on

- Create data, restart API, and read it back.
- Reject duplicate business identifiers using database constraints; roll back failed transactions.

Explain

Why is an async session per request necessary? How are validation schemas different from persisted tables?

Commit checkpoints: C04 at 2:29:48. Exact gates and messages are on the next page.

When to commit

C04 / Pause at 2:29:48 / Persistent sites

Create a site, restart the API, read it back; failed transaction rolls back.

CareReady: feat: persist care sites with async PostgreSQL

Bookly: feat: persist books with async PostgreSQL

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Services and dependency injection

Watch 64 min; build 60-90 min.

Understand

Watch service extraction at 2:29:48, dependency injection at 2:55:53, and handler integration at 3:01:20. Complete the block at 3:33:35 before starting accounts.

Build in CareReady

- Introduce SiteService with an explicit session; keep HTTP parsing and response codes in routers.
- Move create/update/archive rules into service methods and define transaction ownership. Add deterministic pagination with a maximum page size.
- Describe the future ReadinessService contract: create a request, validate its site and placement, apply policy, and compute readiness from required checks. Do not implement the full workflow before relationships.

Bookly parallel

Move book persistence and ISBN-conflict handling into BookService.

Prove before moving on

- Service tests exercise conflicts without relying on route internals.
- API tests verify that domain failures become the intended status codes.

Explain

Which layer commits? How would two parallel tasks behave if they shared one session?

Commit checkpoints: C05 at 2:55:53, C06 at 3:33:35. Exact gates and messages are on the next page.

When to commit

C05 / Pause at 2:55:53 / Service boundary

Business rules are testable outside handlers; no shared session across concurrent work.

CareReady: refactor: extract care site service

Bookly: refactor: extract book catalog service

C06 / Pause at 3:33:35 / Injected services

Handlers use injected sessions/services; pagination is stable and bounded.

CareReady: refactor: inject services and paginate sites

Bookly: refactor: inject services and paginate books

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Users and schema evolution

Watch 69 min; build 90-150 min.

Understand

Learn user models, Alembic, account creation, and password hashing. The video uses passlib; this workspace plans pwdlib with Argon2. Understand the security concept and use the selected maintained dependency consistently.

Build in CareReady

- Create User and OrganizationMembership models; membership will carry tenant role.
- Initialize Alembic, generate and review the first migration, and test upgrade on a clean database.
- Add registration with normalized email uniqueness, a password policy, and a response that never includes password_hash.

Bookly parallel

Create reader accounts and the initial Bookly migrations.

Prove before moving on

- Duplicate email is handled consistently; raw passwords never persist.
- Correct password verifies and incorrect password fails.
- Clean database upgrade succeeds; inspect generated constraints and indexes.

Explain

Why hash rather than encrypt passwords? Why review generated migration code?

Commit checkpoints: C07 at 3:59:57, C08 at 4:42:57. Exact gates and messages are on the next page.

When to commit

C07 / Pause at 3:59:57 / Reviewed schema

User and membership migration upgrades a clean DB; inspect indexes and constraints.

CareReady: feat: add account and membership migrations

Bookly: feat: add reader account migrations

C08 / Pause at 4:42:57 / Account creation

Registration hashes passwords, rejects duplicate normalized email, and never returns a hash.

CareReady: feat: register accounts with Argon2 passwords

Bookly: feat: register reader accounts securely

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Sessions, JWTs, and revocation

Watch 85 min; build 120-180 min.

Understand

Follow JWT setup, login, Bearer validation, refresh, and Redis revocation. Treat authentication as an end-to-end session design rather than token creation alone.

Build in CareReady

- Implement login with short-lived access tokens and explicit issuer, audience, expiry, and token-type checks.
- Store hashed refresh tokens durably; rotate them on use and revoke the token family on reuse.
- Implement logout/revocation with Redis TTLs. Decide how revocation behaves if Redis is unavailable; document and test the chosen policy.

Bookly parallel

Use the same auth concepts for readers and editors, with separate signing secrets per deployed product.

Prove before moving on

- Expired/wrong-audience/wrong-type tokens fail.
- A reused refresh token cannot create a new session; logout makes the session unusable.
- Authentication errors never echo secrets or disclose whether a password was close.

Explain

Why are access and refresh tokens different? What happens when a token is stolen or Redis fails?

Commit checkpoints: C09 at 5:33:14, C10 at 5:50:04, C11 at 6:07:39. Exact gates and messages are on the next page.

When to commit

C09 / Pause at 5:33:14 / Access authentication

Login and Bearer dependency validate expiry, issuer, audience, and token type.

CareReady: feat: authenticate CareReady access tokens

Bookly: feat: authenticate Bookly access tokens

C10 / Pause at 5:50:04 / Rotating refresh sessions

Durable hashed refresh tokens rotate; replay revokes the affected session family.

CareReady: feat: rotate refresh sessions and detect reuse

Bookly: feat: rotate reader refresh sessions

C11 / Pause at 6:07:39 / Revocation

Logout revokes access; Redis TTL and outage policy are tested.

CareReady: feat: revoke sessions with Redis

Bookly: feat: revoke reader sessions with Redis

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Authorization and tenant context

Watch 32 min; build 90-150 min.

Understand

Use the current-user and role-checker concepts, then extend them into organization-scoped memberships. Multi-tenancy is an independent CareReady requirement.

Build in CareReady

- Resolve active membership from the authenticated user and selected organization. Never accept a tenant header without membership validation.
- Add admin, IT operator, approver, employee, and auditor permission rules.
- Apply tenant predicates to every site operation; include tenant identity in cache and job boundaries later.

Bookly parallel

Add reader/editor/moderator permissions plus ownership checks for reader-generated content.

Prove before moving on

- A user from organization A cannot list, read, edit, or delete B records even with a known UUID.
- An employee cannot grant roles; an auditor cannot mutate data.

Explain

Why is checking role alone insufficient? What is the distinction between 401 and 403?

Commit checkpoints: C12 at 6:39:24. Exact gates and messages are on the next page.

When to commit

C12 / Pause at 6:39:24 / Tenant permissions

Two organizations cannot access each other; coordinator/operator/approver/admin/auditor permissions deny by default.

CareReady: feat: enforce tenant memberships and permissions

Bookly: feat: enforce reader editor and moderator permissions

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Relationships and operational workflows

Watch 81 min; build in several 60-90 minute blocks. Expect 5-8 hours for this domain-heavy stage.

Understand

Translate relationships into a coherent operations domain. Do not add every entity in one commit; establish one complete provisioning flow.

Build in CareReady

- Add StaffPlacement, KitTemplate, ReadinessRequest, checklist items, Vendor, Asset, Reservation, Assignment, PurchaseRequest, Approval, Shipment, and ReturnCase in small migrations.
- Reserve available equipment atomically; route stock shortages into an independent purchase approval. Enforce same-tenant references and invalidate approval if the requested amount changes.
- Track draft -> submitted -> approved -> preparing -> dispatched -> ready; rejected/cancelled are explicit alternatives. Derive readiness from delivery and required setup checks.
- Track return -> received -> inspected -> wipe_verified -> closed. Only then release equipment for reuse; record authorized exceptions and prevent duplicate assignment.

Bookly parallel

Build book-review-user and book-tag relationships, then curated collections and moderation ownership.

Prove before moving on

- Cross-tenant relationships fail even if IDs are valid.
- Two concurrent requests cannot assign the same asset.
- An invalid workflow transition returns 409; rejected requests cannot silently ship.

Explain

Which relationships need a join table? Which invariant requires a database constraint or lock?

Commit checkpoints: C13 at 7:05:53, C14 at 7:59:58, C15 at 7:59:58, C16 at 7:59:58, C17 at 7:59:58. Exact gates and messages are on the next page.

When to commit

C13 / Pause at 7:05:53 / Relational foundation

Sites, placements, kit templates, requests, and assets use same-tenant foreign keys; reversed placement dates fail.

CareReady: feat: model placements kits and readiness requests

Bookly: feat: relate books readers and reviews

C14 / Pause at 7:59:58 / Inventory reservation

After finishing this watch block, pause for multiple build checkpoints. Competing requests cannot reserve the same device; expired reservations release safely.

CareReady: feat: reserve equipment atomically

Bookly: feat: add book tags and curated collections

C15 / Pause at 7:59:58 / Approval policy

At the same pause: requester cannot approve own purchase; changing amount invalidates approval.

CareReady: feat: enforce independent purchase approvals

Bookly: no forced equivalent. Continue its relevant catalog/review work separately.

C16 / Pause at 7:59:58 / Delivery and readiness

At the same pause: delayed delivery or an incomplete checklist prevents ready status; duplicate shipment events do not repeat side effects.

CareReady: feat: derive staff readiness from fulfillment checks

Bookly: no forced equivalent. Continue its relevant catalog/review work separately.

C17 / Pause at 7:59:58 / Return and reuse

At the same pause: missing inspection or wipe evidence blocks reuse; authorized exceptions have a reason.

CareReady: feat: gate equipment reuse on return evidence

Bookly: no forced equivalent. Continue its relevant catalog/review work separately.

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Error contracts, logs, and audit evidence

Watch 65 min; build 90-150 min.

Understand

Follow exception types, handlers, middleware, CORS, and trusted hosts. Add audit behavior as an independent product requirement.

Build in CareReady

- Define stable error codes for not-found, conflict, invalid transition, and insufficient permission.
- Add correlation IDs and structured request logs; omit tokens, passwords, and unnecessary employee details.
- Write append-only audit events in the same transaction as state changes. Configure explicit allowed origins/hosts for deployment.

Bookly parallel

Add consistent catalog/review errors and moderation audit events.

Prove before moving on

- Unexpected failures return safe 500 responses while logs retain a correlation ID.
- Failed transactions do not leave a misleading success audit record.
- Tenant-scoped audit export never includes another organization.

Explain

How does operational logging differ from durable audit history? Why is CORS not authorization?

Commit checkpoints: C18 at 8:33:25, C19 at 9:05:04. Exact gates and messages are on the next page.

When to commit

C18 / Pause at 8:33:25 / Domain errors

Translate invalid transition, stale approval, and stock conflict into safe stable errors.

CareReady: feat: add readiness domain error contracts

Bookly: feat: add catalog and review error contracts

C19 / Pause at 9:05:04 / Logs and evidence

Correlation IDs, explicit hosts/origins, and audit events work; mutation and audit commit together.

CareReady: feat: record transactional audit events and request traces

Bookly: feat: audit moderation and trace requests

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Email and account recovery

Watch 96 min; build 90-150 min.

Understand

Understand transport setup, verification, and password reset. Keep templates simple and use a local capture service or provider sandbox while building.

Build in CareReady

- Define a mail adapter so business logic is independent of a provider.
- Add single-use, expiring verification and reset tokens; store only safe token representations.
- Use generic reset responses to reduce account enumeration; invalidate relevant sessions after reset. Do not send real employee mail from test fixtures.

Bookly parallel

Add verification and account recovery for reader accounts.

Prove before moving on

- Expired and reused reset links fail.
- Reset responses do not reveal account existence.
- A failed mail send is observable and safely retryable.

Explain

What can a reset token authorize? Why must retries not generate duplicate side effects?

Commit checkpoints: C20 at 9:31:21, C21 at 10:07:51, C22 at 10:40:38. Exact gates and messages are on the next page.

When to commit

C20 / Pause at 9:31:21 / Mail adapter

Use local capture or provider sandbox; transport failures can be simulated without real recipients.

CareReady: feat: add a testable notification mail adapter

Bookly: feat: add a testable reader mail adapter

C21 / Pause at 10:07:51 / Account verification

Confirmation tokens expire and are single-use.

CareReady: feat: verify staff accounts with expiring tokens

Bookly: feat: verify reader accounts

C22 / Pause at 10:40:38 / Account recovery

Generic reset response; expired/reused tokens fail; reset invalidates relevant sessions.

CareReady: feat: add safe account recovery

Bookly: feat: add safe reader account recovery

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Durable background work

Watch 43 min; build 120-180 min.

Understand

Compare in-process tasks with Celery/Redis and worker monitoring. Use in-process tasks only for short, non-critical work.

Build in CareReady

- Create Celery jobs for approaching start dates, late shipments, expiring reservations, and overdue returns.
- Pass organization and record identifiers; reload state and check tenant scope inside the worker.
- Add retry/backoff, idempotency keys, and an outbox so committing a request cannot lose its notification. Keep Flower private and authenticated when deployed.

Bookly parallel

Add catalog-import or review-notification jobs with the same reliability principles.

Prove before moving on

- Kill and restart a worker; a retry does not duplicate a shipment or message.
- A job for A cannot access B; permanently failed jobs are visible for review.

Explain

Why can an in-process task disappear? What does at-least-once delivery require from your code?

Commit checkpoints: C23 at 10:45:44, C24 at 11:16:17, C25 at 11:23:48. Exact gates and messages are on the next page.

When to commit

C23 / Pause at 10:45:44 / Task boundary

Only disposable post-response work uses BackgroundTasks; document crash-loss behavior.

CareReady: chore: define critical and disposable task boundaries

Bookly: chore: define notification task boundaries

C24 / Pause at 11:16:17 / Durable jobs

Celery handles due-date, late-shipment, and return reminders; tenant context, outbox, retry/backoff, and idempotency are tested.

CareReady: feat: queue reliable readiness and return jobs

Bookly: feat: queue reliable catalog and review jobs

C25 / Pause at 11:23:48 / Worker visibility

Flower is private; failed/retried jobs are visible; worker restart does not duplicate effects.

CareReady: ops: monitor worker retries and failures

Bookly: ops: monitor catalog worker jobs

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

API contracts and test depth

Watch 45 min; build 2-4 hours.

Understand

Use OpenAPI examples, unit tests, mocks, and schema-driven tests. Build on tests already written throughout the guide; do not defer all testing until this point.

Build in CareReady

- Document auth, pagination, transitions, errors, and representative fictional payloads.
- Separate pure service tests from real PostgreSQL/Redis integration tests and API tests.
- Add a two-tenant security matrix and concurrency tests. Use Schemathesis against a disposable test service, never production data.

Bookly parallel

Test search/filtering, review ownership, duplicate ISBNs, moderation, and authentication.

Prove before moving on

- CI runs from a clean clone; no test depends on the developer database.
- A deliberately broken tenant filter makes the security tests fail.
- Mock-based tests are complemented by real dependency behavior.

Explain

What failure could pass a mock test and still occur against PostgreSQL?

Commit checkpoints: C26 at 11:36:02, C27 at 12:01:27, C28 at 12:09:17. Exact gates and messages are on the next page.

When to commit

C26 / Pause at 11:36:02 / API documentation

Document auth, pagination, transition rules, examples, and failure responses in OpenAPI.

CareReady: docs: describe staff readiness API contracts

Bookly: docs: describe book discovery API contracts

C27 / Pause at 12:01:27 / Behavior tests

Run service/API and real PostgreSQL/Redis tests, tenant isolation, races, and refresh replay cases in CI.

CareReady: test: verify tenancy reservations and session security

Bookly: test: verify reviews permissions and sessions

C28 / Pause at 12:09:17 / Contract tests

Schemathesis runs against disposable fixtures; deliberately broken validation produces a failing test.

CareReady: test: exercise OpenAPI contracts with Schemathesis

Bookly: test: exercise Bookly OpenAPI contracts

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

Deployment and operations

Watch 44 min; build 2-4 hours plus hardening.

Understand

Observe the deployment lifecycle and adapt it to current provider documentation and your budget. This starter has no hosted deployment or paid resources.

Build in CareReady

- Build an API image and a separate worker process; configure managed secrets and private database/cache access.
- Run reviewed migrations once per release, then smoke-test readiness and core workflows.
- Add backup/restore drills, safe rollback, logs/metrics, and alerting. Restrict CORS/hosts, protect operational dashboards, and validate token settings.

Bookly parallel

Deploy Bookly independently with its own credentials, database, and release checks.

Prove before moving on

- A clean deployment passes smoke tests without manual database edits.
- A failed dependency removes readiness but leaves process liveness intact.
- Restore a backup into a disposable database and verify representative records.

Explain

How will you roll back code without corrupting a migrated schema? How do you know a worker is stuck?

Commit checkpoints: C29 at 12:52:54. Exact gates and messages are on the next page.

When to commit

C29 / Pause at 12:52:54 / Release and restore

Build API/worker images; validate release migrations, secrets, health, rollback, and a restore drill before declaring deployed.

CareReady: ops: document and verify CareReady deployment

Bookly: ops: document and verify Bookly deployment

Record the evidence

After the checks pass, record the actual date, commit SHA, and one sentence of proof in docs/PROGRESS.md. Suggested messages describe future work; do not use them until the behavior exists.

Run make test and make lint, inspect git diff, stage only related files, commit, then push. If the behavior needs several commits, use accurate smaller messages.

The technology-to-product map

Technology	CareReady behavior to prove
FastAPI / Pydantic	Site and placement inputs, kit requests, typed responses, filters, OpenAPI.
SQLModel / PostgreSQL / async	Tenant-owned records, transaction-safe reservations, per-request sessions.
Alembic / services / DI	Reviewed schema upgrades and testable workflow boundaries.
JWT / refresh / hashing	Argon2 passwords, validated access tokens, rotation and replay detection.
Redis / RBAC / tenancy	Revocation TTLs; tenant-aware caches and jobs; role/ownership checks.
Relationships / state machines	Placement-to-kit-to-device flow; approvals, shipments, return evidence.
Errors / middleware / audit	Safe contracts, request IDs, explicit CORS/hosts, atomic audit history.
Email / account recovery	Sandbox delivery, verification, expiring single-use reset links.
BackgroundTasks / Celery / Flower	Disposable tasks vs durable deadline/return jobs; retries and monitoring.
pytest / mocks / Schemathesis	Unit, real-service, API, contract, security, and concurrency tests.
Docker / CI / deployment	API and worker releases, migration steps, health, backup/restore.

Measure later; do not invent results

Define ready-by-start-date percentage, overdue-return rate, reused-stock share, and request-to-ready time from recorded events. Add a tenant-scoped Redis dashboard cache only after measuring useful queries; invalidate it after mutations.

What you must be able to demonstrate

- Create Cedar Care and Harbour Care, users with distinct roles, sites, and staff placements using fictional data.
- Submit a kit request, reserve stock or request purchase approval, track delivery/setup, and derive day-one readiness.
- Show self-approval, changed-price approval, double reservation, and cross-tenant requests being rejected.
- Close a placement, receive and inspect equipment, record wipe evidence, then release it for reuse.
- Retry a reminder and replay a shipment event without duplicate effects. Show a worker restart and its recovery.
- Run migrations on a clean database, execute CI, deploy under controlled settings, and restore a backup into a disposable environment.
- For Bookly: search/paginate a catalog, manage reviews with ownership checks, moderate content, and demonstrate cache/job behavior.

Explain the whole path

HTTP -> validation -> authenticated principal -> tenant membership and permissions -> service transaction -> database constraints -> audit/outbox -> response -> idempotent worker. Explain where a failure rolls back and how a retry remains safe.

Scope discipline

Start with laptops and a small set of accessories. Manual vendor/shipment adapters are enough for the MVP. Avoid expanding into clinical data, live device control, payments, or AI before the transaction and isolation rules are demonstrably correct.

Portfolio readiness

Update the READMEs with implemented behavior and evidence. Keep limitations explicit. A starter and a list of installed libraries are not completed products. Use measured results only after you can reproduce the measurement.

Chapter -> independent application

These are the original video positions with paraphrased topic names and CareReady exercises. Use the stage/checkpoint pages for tests and commit gates.

Timestamp	Concept	CareReady application
0:00	Orientation	Define the provisioning story
0:01:00	Workspace setup	Use the two isolated repositories
0:07:30	First HTTP server	Explain the existing health route
0:10:45	Server commands	Run each API on its own port
0:14:11	URL identifiers	Site UUID detail route
0:17:23	Request tooling	Save repeatable site requests
0:20:58	Query inputs	Filter active sites
0:24:40	Combined inputs	Site detail plus optional fields
0:26:51	Optional filters	Define safe filter defaults
0:31:48	JSON validation	SiteCreate plus placement-date validation
0:39:11	Header handling	Understand request metadata
0:49:43	Memory-backed CRUD	Site create/read/update/delete
1:23:37	Router modules	Move site routes out of main
1:38:22	ORM introduction	Map care sites to persisted models
1:42:33	Database preparation	Start isolated PostgreSQL
1:44:13	Typed settings	Read database configuration safely
1:53:38	Async ORM wiring	Use per-request async sessions

Chapter -> independent application

These are the original video positions with paraphrased topic names and CareReady exercises. Use the stage/checkpoint pages for tests and commit gates.

Timestamp	Concept	CareReady application
1:58:38	Resource lifetime	Own clients in app lifespan
2:10:02	Persisted models	Define Site identity and fields
2:20:00	Table creation	Distinguish prototype schema from migrations
2:27:08	ORM operations	Persist and retrieve a site
2:29:48	Service boundary	Extract SiteService
2:55:53	Dependency patterns	Inject sessions and principal context
3:01:20	Handler integration	Keep routes thin and explicit
3:33:35	Account model	User plus organization membership
3:42:09	Schema migrations	Review Alembic revisions
3:59:57	Registration logic	Normalize and validate accounts
4:18:55	Password storage	Argon2 through pwdlib in this plan
4:25:42	Registration route	Return safe account fields
4:42:57	Token concepts	Define access and refresh responsibilities
4:48:29	JWT library	Validate issuer audience and expiry
5:01:13	Sign-in flow	Verify credentials and issue sessions
5:13:59	Bearer dependency	Reject invalid principals
5:33:14	Session renewal	Rotate durable refresh tokens

Chapter -> independent application

These are the original video positions with paraphrased topic names and CareReady exercises. Use the stage/checkpoint pages for tests and commit gates.

Timestamp	Concept	CareReady application
5:50:04	Revocation store	Redis TTL and logout behavior
6:07:39	Permission design	Organization-scoped roles
6:09:45	Principal lookup	Resolve active membership
6:20:25	Role persistence	Roles on membership not global user
6:26:55	Permission dependency	Deny unauthorized operations
6:39:24	Relational modeling	Staff placements, kits, requests, and assets
7:05:53	Relationship expansion	Reservations, approvals, shipments, returns
7:59:58	Failure semantics	Consistent safe API errors
8:04:06	Domain error types	ReadinessConflict and InvalidTransition
8:18:14	Exception translation	Map domain errors to HTTP
8:23:26	Handler registration	Apply errors across routers
8:33:25	Middleware lifecycle	Understand request ordering
8:36:54	Request logging	Correlation IDs without secrets
8:53:28	Middleware practice	Measure latency and failure paths
8:59:36	ASGI integration	Check middleware compatibility
9:00:21	Origin policy	Allow only intended frontends
9:03:34	Host policy	Configure deployed hostnames

Chapter -> independent application

These are the original video positions with paraphrased topic names and CareReady exercises. Use the stage/checkpoint pages for tests and commit gates.

Timestamp	Concept	CareReady application
9:05:04	Mail requirements	Define notification adapter
9:06:39	Mail configuration	Use sandbox credentials
9:21:46	Delivery check	Send only controlled test messages
9:31:21	Account confirmation	Single-use verification token
10:07:51	Recovery flow	Expiring reset plus session invalidation
10:40:38	Deferred work	Separate deadline jobs from disposable tasks
10:43:00	In-process tasks	Use only for non-critical work
10:45:44	Queue and worker	Tenant-scoped deadline and return jobs
11:16:17	Worker visibility	Monitor retries without public dashboards
11:23:48	OpenAPI presentation	Document operation contracts
11:36:02	Test strategy	Unit integration and API layers
11:37:10	Mocks and pytest	Test business decisions and failures
12:01:27	Schema-based tests	Exercise disposable API contracts
12:09:17	Hosted release	Secrets migrations health and rollback

Run CareReady and Bookly

CareReady repository: <https://github.com/PraneetS11/careready-api> Bookly repository: <https://github.com/PraneetS11/bookly-api>

CareReady - installed environment

From the workspace, run:

```
cd careready-api
```

```
source .venv/bin/activate
```

```
make services
```

```
make run
```

Open <http://127.0.0.1:8002/docs>. In another terminal in the same repository: `make test` and `make lint`. To recreate the environment, use the README quick start and `requirements.lock`.

Bookly - separate terminal

```
cd bookly-api
```

```
source .venv/bin/activate
```

```
make services
```

```
make run
```

Open <http://127.0.0.1:8001/docs>. Each repository has its own `.git`, `.venv`, `.env`, PostgreSQL database/volume, Redis service, and commit history.

Service	Bookly	CareReady
API	127.0.0.1:8001	127.0.0.1:8002
PostgreSQL	127.0.0.1:5433	127.0.0.1:5434
Redis	127.0.0.1:6381	127.0.0.1:6382

References

<https://www.youtube.com/watch?v=TO4aQ3ghFOc>

<https://github.com/jod35/fastapi-beyond-CRUD>

<https://jod35.github.io/fastapi-beyond-crud-docs/site/>

<https://fastapi.tiangolo.com/tutorial/testing/>

<https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>